QuickQuiz TDD evidence

Tongxiang Wang (Leon) - AUT ID 24245808

User stories #72 Sudden Death and #73 Solo Practice

This document records six genuine test-first development cycles from the supplied Leon branch. Each new case failed before its behaviour was implemented, then passed; earlier cases remained passing. The final suite has 26 passing tests.

Execution source: this assisted development session in the Codex workspace. Source/output images below are screenshots of a report rendering the captured stage records, not earlier student-PC IDE screenshots. The raw source, complete transcripts and timestamps are included in docs/tdd/stages. No historic student Git commits, teammate acceptance or TA sign-off are claimed.

Check	Result
TDD stages	Six RED stages failed as expected; six GREEN stages passed
Final unit tests	26 passed, 0 failed
Production build and lint	Both commands exited successfully
Browser checks	Practice, normal turns, tie-break repeat/winner, login and logout cancel passed using local-bank API fixtures
Independent acceptance	Hafsa (#72) and Tim (#73): pending actual manual checks
TA lab sign-off	Pending the Week 12 demonstration

Assessment requirements: Assessment 2 - Final Deliverables, page 3; COMP602 Laboratory 08, pages 1-2. Lab 08 specifies at least three cases per member story. The team PDF is uploaded to Documentation / Test-Driven Development. The coordinator collects each member's real contribution.

Story #72 Sudden Death

As a player, I want a sudden-death question if we draw so that every game has a winner.

Assigned developer: Leon. Assigned independent tester: Hafsa.

1. Given the two normal sets end in a draw, when results would normally open, then a sudden-death question is shown.

2. Given sudden death has a winner, when final results open, then that winner is shown instead of a tie.

3. Implemented additions: both players answer the same question with separate timers; answers remain private until both finish; identical outcomes repeat; timeout is incorrect. Confirm these choices with the PO.

Case	Development behaviour
T01	a completed tied game requires sudden death
T02	only player one correct gives player one the win
T03	the same result for both players requires another question

Evidence order for each case: test code, failing runner output, implementation source, passing runner output. Captioned excerpts preserve the actual captured source/output; complete records are supplied in JSON. The other framework imports and test setup are in src/tests.

T01: a completed tied game requires sudden death

RED: the new tie-break predicate returned false. GREEN: require a completed two-player game with equal total scores. The final refactor uses the shared game configuration constant.

1. Test code from the RED stage

```
test('T01: a completed tied game requires sudden death', () => {
  const game = makeGame()
  completeNormalGame(game, 2, 2)
  assert.equal(game.session.needsSuddenDeath(2), true)
})
```

2. Actual failing Node.js test-runner output (excerpt)

```
✖ T01: a completed tied game requires sudden death (2.725467ms)
ℹ tests 1
ℹ suites 0
ℹ pass 0
ℹ fail 1
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 113.238575

✖ failing tests:

test at src/tests/suddenDeath.test.js:5:1
✖ T01: a completed tied game requires sudden death (2.725467ms)
  AssertionError [ERR_ASSERTION]: Expected values to be strictly equal:

  false !== true

[Stack trace omitted; full transcript is in the stage JSON.]
```

T01 implementation and passing result

3. Captured implementation: src/models/GameSession.js

```
needsSuddenDeath(totalChallenges = 2) {
    return this.players.length === 2 &&
this.isGameComplete(totalChallenges) &&
      this.players[0].getScore() === this.players[1].getScore()
  }
```

4. Actual passing Node.js test-runner output

```
✔ T01: a completed tied game requires sudden death (1.345543ms)
ℹ tests 1
ℹ suites 0
ℹ pass 1
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 100.856442
```

The complete stage includes all earlier tests for this story. GREEN is successful only when those earlier cases also pass. Replay this case with: npm run test:tdd -- T01

T02: only player one correct gives player one the win

RED: the new round always reported waiting. GREEN: after both results exist, different correctness values identify the correct player as winner.

1. Test code from the RED stage

```
test('T02: only player one correct gives player one the win', () => {
  const game = makeGame()
  const question = game.repository.getCategories()[0].getQuestions()[0]
  const round = new SuddenDeathRound(game.session.players, question)
  round.submitResult('player-1', true)
  round.submitResult('player-2', false)
  assert.deepEqual(round.getOutcome(), { state: 'winner', winnerId:
'player-1' })
})
```

2. Actual failing Node.js test-runner output (excerpt)

```
✔ T01: a completed tied game requires sudden death (1.399545ms)
✖ T02: only player one correct gives player one the win (1.757755ms)
ℹ tests 2
ℹ suites 0
ℹ pass 1
ℹ fail 1
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 87.086761

✖ failing tests:

test at src/tests/suddenDeath.test.js:12:1
✖ T02: only player one correct gives player one the win (1.757755ms)
  AssertionError [ERR_ASSERTION]: Expected values to be strictly deep-
equal:
  + actual - expected

    {
  +   state: 'waiting',
  +   winnerId: null
  -   state: 'winner',
  -   winnerId: 'player-1'
    }


[Additional lines omitted; see the original stage JSON.]
```

T02 implementation and passing result

3. Captured implementation: src/models/SuddenDeathRound.js

```
// developer-Leon: both players answer one question before a winner is
decided.
export default class SuddenDeathRound {
  constructor(players, question, roundNumber = 1) {
    this.playerIds = players.map((player) => player.playerId)
    this.question = question
    this.roundNumber = roundNumber
    this.results = {}
  }

  submitResult(playerId, isCorrect) {
    this.results[playerId] = isCorrect
  }

  getOutcome() {
    const [first, second] = this.playerIds
    if (Object.hasOwn(this.results, first) && Object.hasOwn(this.results,
second) &&
      this.results[first] !== this.results[second]) {
      return { state: 'winner', winnerId: this.results[first] ? first :
second }
    }
    return { state: 'waiting', winnerId: null }
  }
}
```

4. Actual passing Node.js test-runner output

```
✔ T01: a completed tied game requires sudden death (1.033832ms)
✔ T02: only player one correct gives player one the win (0.741941ms)
ℹ tests 2
ℹ suites 0
ℹ pass 2
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 96.945922
```

The complete stage includes all earlier tests for this story. GREEN is successful only when those earlier cases also pass. Replay this case with: npm run test:tdd -- T02

T03: the same result for both players requires another question

RED: equal correctness values still reported waiting. GREEN: both equal values report repeat so a new question can be started.

1. Test code from the RED stage

```
test('T03: the same result for both players requires another question', ()
=> {
  const game = makeGame()
  const question = game.repository.getCategories()[0].getQuestions()[0]
  const round = new SuddenDeathRound(game.session.players, question)
  round.submitResult('player-1', false)
  round.submitResult('player-2', false)
  assert.deepEqual(round.getOutcome(), { state: 'repeat', winnerId: null })
})
```

2. Actual failing Node.js test-runner output (excerpt)

```
✔ T01: a completed tied game requires sudden death (1.080071ms)
✔ T02: only player one correct gives player one the win (1.015684ms)
✖ T03: the same result for both players requires another question
(1.348497ms)
ℹ tests 3
ℹ suites 0
ℹ pass 2
ℹ fail 1
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 94.247541

✖ failing tests:

test at src/tests/suddenDeath.test.js:21:1
✖ T03: the same result for both players requires another question
(1.348497ms)
  AssertionError [ERR_ASSERTION]: Expected values to be strictly deep-
equal:
  + actual - expected

    {
  +   state: 'waiting',
  -   state: 'repeat',
      winnerId: null
    }


[Additional lines omitted; see the original stage JSON.]
```

T03 implementation and passing result

3. Captured implementation: src/models/SuddenDeathRound.js

```
// developer-Leon: both players answer one question before a winner is
decided.
export default class SuddenDeathRound {
  constructor(players, question, roundNumber = 1) {
    this.playerIds = players.map((player) => player.playerId)
    this.question = question
    this.roundNumber = roundNumber
    this.results = {}
  }

  submitResult(playerId, isCorrect) {
    this.results[playerId] = isCorrect
  }

  getOutcome() {
    const [first, second] = this.playerIds
    if (Object.hasOwn(this.results, first) && Object.hasOwn(this.results,
second)) {
      if (this.results[first] === this.results[second]) {
        return { state: 'repeat', winnerId: null }
      }
      return { state: 'winner', winnerId: this.results[first] ? first :
second }
    }
    return { state: 'waiting', winnerId: null }
  }
}
```

4. Actual passing Node.js test-runner output

```
✔ T01: a completed tied game requires sudden death (1.248456ms)
✔ T02: only player one correct gives player one the win (0.653157ms)
✔ T03: the same result for both players requires another question
(0.210378ms)
ℹ tests 3
ℹ suites 0
ℹ pass 3
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 98.287945
```

The complete stage includes all earlier tests for this story. GREEN is successful only when those earlier cases also pass. Replay this case with: npm run test:tdd -- T03

Story #73 Solo Practice

As a player, I want a solo practice mode so that I can train before I play someone else.

Assigned developer: Leon. Assigned independent tester: Tim.

1. Given the dashboard, when Practice is chosen, then the quiz starts with one player, no Player 2 and no handoff.

2. Given practice, when a category is picked, then its questions and the normal timer are used.

3. Implemented addition: personal results and saved history identify Practice rather than a competitive victory.

Case	Development behaviour
S01	practice creates a game with only the current player
S02	practice loads the chosen category with the normal timer
S03	a completed practice shows a personal result without a handoff

Evidence order for each case: test code, failing runner output, implementation source, passing runner output. Captioned excerpts preserve the actual captured source/output; complete records are supplied in JSON. The other framework imports and test setup are in src/tests.

S01: practice creates a game with only the current player

RED: the original constructor rejected a single player. GREEN: validate one player for practice and two for versus, and store the explicit mode.

1. Test code from the RED stage

```
test('S01: practice creates a game with only the current player', () => {
  const game = makeGame('practice')
  assert.equal(game.session.mode, 'practice')
  assert.equal(game.session.players.length, 1)
  assert.equal(game.session.activePlayerId, 'player-1')
})
```

2. Actual failing Node.js test-runner output (excerpt)

```
✖ S01: practice creates a game with only the current player (0.74147ms)
ℹ tests 1
ℹ suites 0
ℹ pass 0
ℹ fail 1
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 84.992628

✖ failing tests:

test at src/tests/soloPractice.test.js:5:1
✖ S01: practice creates a game with only the current player (0.74147ms)
  Error: GameSession requires exactly two players.

[Stack trace omitted; full transcript is in the stage JSON.]
```

S01 implementation and passing result

3. Captured implementation: src/models/GameSession.js

```
constructor(sessionId, players, mode = 'versus') {
    if (!sessionId) {
      throw new Error('GameSession requires a sessionId.')
    }

    if (!['versus', 'practice'].includes(mode)) {
      throw new Error('Unknown game mode.')
    }
    const requiredPlayers = mode === 'practice' ? 1 : 2
    if (!Array.isArray(players) || players.length !== requiredPlayers) {
      throw new Error(`GameSession requires exactly ${requiredPlayers}
players.`)
    }

    this.mode = mode // developer-Leon
```

4. Actual passing Node.js test-runner output

```
✔ S01: practice creates a game with only the current player (1.03257ms)
ℹ tests 1
ℹ suites 0
ℹ pass 1
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 93.740438
```

The complete stage includes all earlier tests for this story. GREEN is successful only when those earlier cases also pass. Replay this case with: npm run test:tdd -- S01

S02: practice loads the chosen category with the normal timer

RED: the practice entry-point stub returned null. GREEN: select the category and delegate to the existing timed question-set method.

1. Test code from the RED stage

```
test('S02: practice loads the chosen category with the normal timer', () =>
{
  const game = makeGame('practice')
  const questions =
game.repository.getCategory('science').getQuestions().slice(0, 3)
  const first = game.controller.startPractice('science', questions)
  assert.equal(first, questions[0])
  assert.equal(game.session.selectedCategoryId, 'science')
  assert.equal(game.session.getCurrentRound().timeLimitSeconds, 15)
})
```

2. Actual failing Node.js test-runner output (excerpt)

```
✔ S01: practice creates a game with only the current player (0.924837ms)
✖ S02: practice loads the chosen category with the normal timer
(1.281186ms)
ℹ tests 2
ℹ suites 0
ℹ pass 1
ℹ fail 1
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 87.293912

✖ failing tests:

test at src/tests/soloPractice.test.js:12:1
✖ S02: practice loads the chosen category with the normal timer
(1.281186ms)
  AssertionError [ERR_ASSERTION]: Expected values to be strictly equal:
  + actual - expected

  + null
  - Question {
  -   categoryId: 'science',
  -   correctAnswer: 'Jupiter',
  -   options: [
  -     'Earth',
  -     'Mars',

[Additional lines omitted; see the original stage JSON.]
```

S02 implementation and passing result

3. Captured implementation: src/controllers/QuizAndTimerController.js

```
startPractice(categoryId, questions) {
    if (this.gameSession.mode !== 'practice') {
      throw new Error('Practice requires a single-player session.')
    }
    this.gameSession.selectCategory(categoryId)
    return this.startPlayerQuestionSet(this.gameSession.activePlayerId,
questions)
  }

  // developer-Taraneh
  // first player in the set answers their 3
```

4. Actual passing Node.js test-runner output

```
✔ S01: practice creates a game with only the current player (0.889624ms)
✔ S02: practice loads the chosen category with the normal timer
(0.296488ms)
ℹ tests 2
ℹ suites 0
ℹ pass 2
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 95.26056
```

The complete stage includes all earlier tests for this story. GREEN is successful only when those earlier cases also pass. Replay this case with: npm run test:tdd -- S02

S03: a completed practice shows a personal result without a handoff

RED: finishing practice tried to finish a two-player challenge that did not exist. GREEN: complete the solo session and return a practiceComplete outcome without handoff.

1. Test code from the RED stage

```
test('S03: a completed practice shows a personal result without a handoff',
() => {
  const game = makeGame('practice')
  const questions =
game.repository.getCategory('science').getQuestions().slice(0, 3)
  game.controller.startPractice('science', questions)
  for (const question of questions) {
    game.controller.checkAnswer('player-1', question.correctAnswer, 10)
    game.controller.continueIfMoreQuestions()
  }
  const outcome = game.controller.finishActivePlayerTurn()
  assert.equal(outcome.practiceComplete, true)
  assert.equal(game.session.roundState, 'completed')
  assert.equal(game.results.getFinalResults().players[0].score, 3)
})
```

2. Actual failing Node.js test-runner output (excerpt)

```
✔ S01: practice creates a game with only the current player (0.97967ms)
✔ S02: practice loads the chosen category with the normal timer
(0.253543ms)
✖ S03: a completed practice shows a personal result without a handoff
(0.525344ms)
ℹ tests 3
ℹ suites 0
ℹ pass 2
ℹ fail 1
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 103.115237

✖ failing tests:

test at src/tests/soloPractice.test.js:21:1
✖ S03: a completed practice shows a personal result without a handoff
(0.525344ms)
  Error: There is no active challenge.

[Stack trace omitted; full transcript is in the stage JSON.]
```

S03 implementation and passing result

3. Captured implementation: src/controllers/QuizAndTimerController.js

```
finishActivePlayerTurn() {
    const round = this.gameSession.getCurrentRound()

    if (!round) {
      throw new Error('There is no active question set.')
    }

    if (!round.isComplete()) {
      throw new Error('Answer all questions before finishing the turn.')
    }
    if (this.gameSession.mode === 'practice') {
      this.gameSession.finishRound()
      return { practiceComplete: true, bothPlayersFinished: true }
    }

    return this.gameSession.completePlayerTurn(
      this.gameSession.activePlayerId,
      round.getCorrectCount(),
    )
  }

  // developer-Hafsa
  // mark right/wrong and keep the correct answer for feedback
```

4. Actual passing Node.js test-runner output

```
✔ S01: practice creates a game with only the current player (0.958698ms)
✔ S02: practice loads the chosen category with the normal timer
(0.296237ms)
✔ S03: a completed practice shows a personal result without a handoff
(0.668881ms)
ℹ tests 3
ℹ suites 0
ℹ pass 3
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 118.354726
```

The complete stage includes all earlier tests for this story. GREEN is successful only when those earlier cases also pass. Replay this case with: npm run test:tdd -- S03

Final verification and refactor

After the six cycles, the behaviours were connected to the React UI. Shared createGame builds the correct player count; the controller handles practice completion and private tie-break turns; results and history distinguish practice. Extra checks cover repeat rounds, player-two wins, early/duplicate answers, timeouts, original turns, fresh state and API fallback.

Command	Observed result
npm test	26 passed, 0 failed
npm run build	Exit code 0
npm run lint	Exit code 0
npm run test:tdd	All twelve recorded stages reproduced the expected result

Runtime: v24.19.0. Unit tests use Node.js's built-in node:test and node:assert. The full final runner output and command records are supplied in docs/tdd/final-verification.json and final-unit-output.txt.

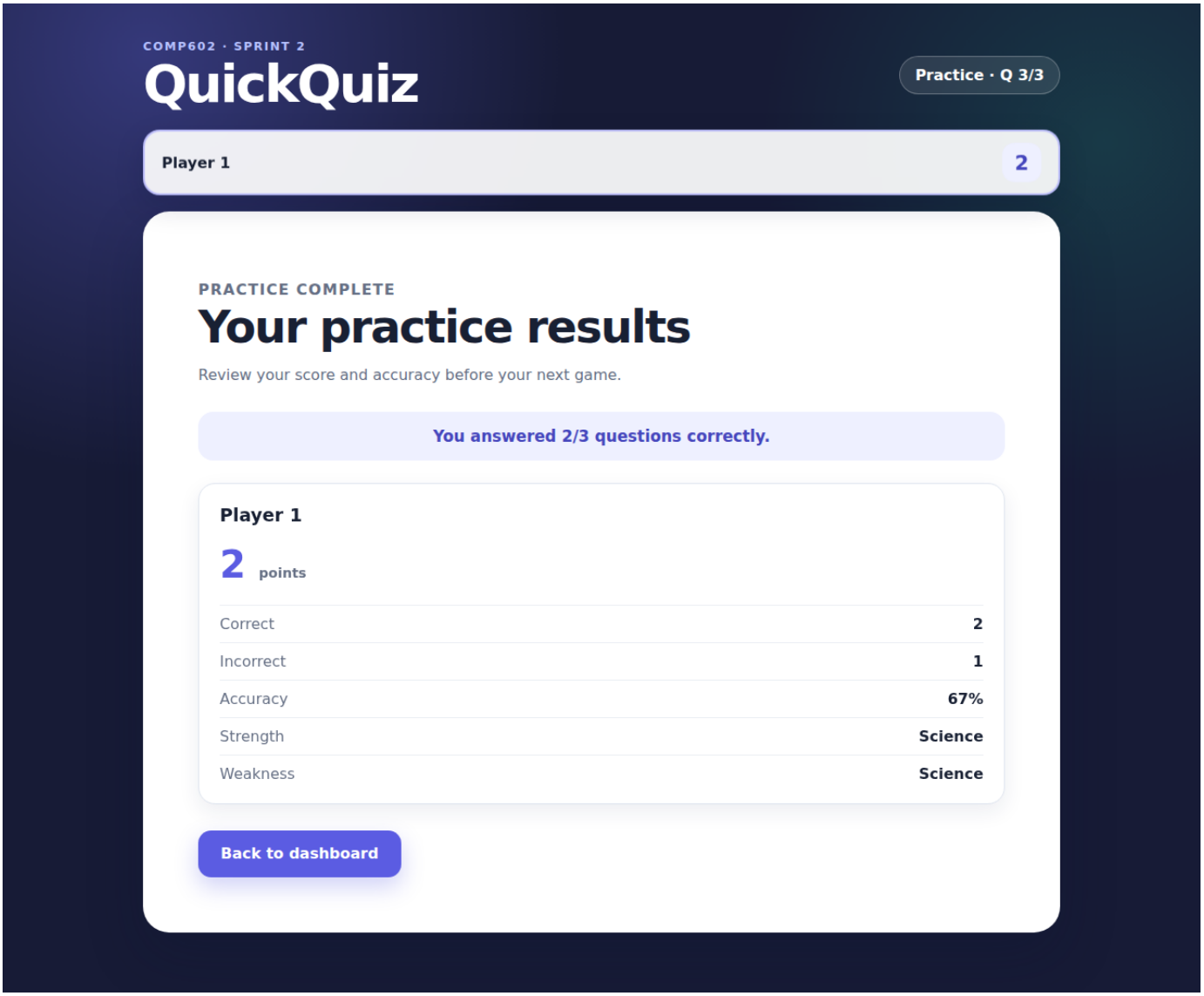
Browser verification used controlled local-bank API responses. It confirms the complete frontend flows but does not assert live Open Trivia Database availability. Unit regression tests exercise the local fallback when the API fails.

To demonstrate locally: npm install, npm test, then npm run test:tdd -- T01 (or another case). The replay runs in temporary folders and does not overwrite final source. Run npm run dev to demonstrate both stories. Add your own IDE/terminal screenshots for the lab demonstration where required.

Other members: collect their genuine story/test/source/failure/pass records. Existing logout confirmation was regression checked; this upload does not supply its original TDD chronology. Team checklist and manual acceptance forms are included. The final code and tests still need integration into the team's Release branch.

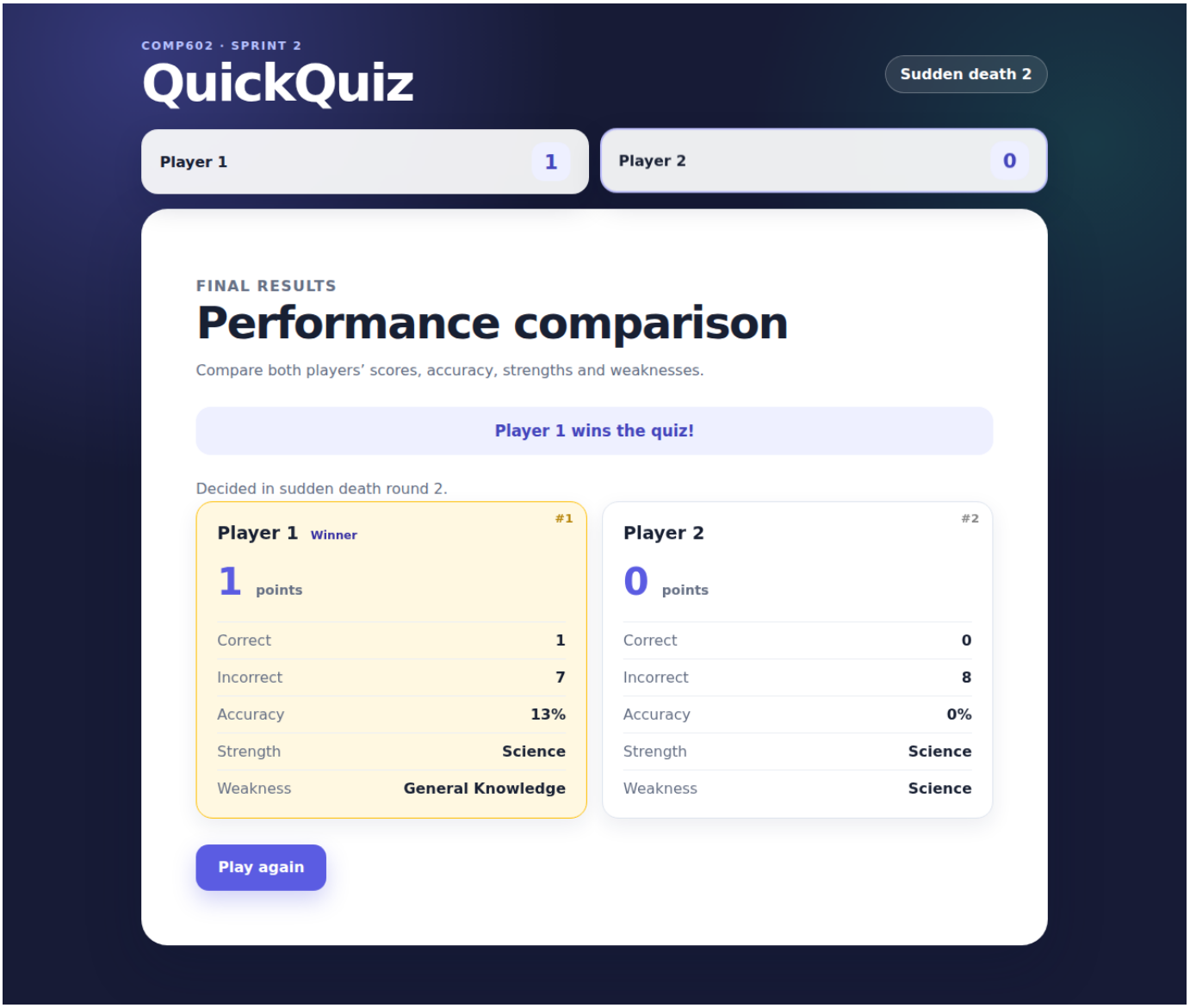
Browser acceptance evidence: solo practice

Automated Chromium run: the dashboard Practice entry starts one player, preserves the timer/feedback, completes three questions and displays a personal result. The recorded fixture run answered two correctly and saved the attempt as Practice.



Browser acceptance evidence: sudden death

Automated Chromium run: both normal sets ended tied. The first tie-break remained tied, so another question opened. Only player one answered correctly in the second tie-break; final results displayed Player 1 as the winner. Correct answers were hidden during the handoff.



Original screenshots and the recorded browser verification are in docs/tdd/ui. These automated checks do not substitute for Hafsa/Tim's actual acceptance or TA sign-off.